Week 4 Decision Gate: *King Octopus and the servants*

Jeremy Evert

September 8, 2026

Source and Problem Statement

The following puzzle is from Puzzle Prime:

King Octopus and His Servants

King Octopus has servants with 6, 7, or 8 legs. The servants with 7 legs always lie, and the servants with 6 or 8 legs always tell the truth. One day four of the king's servants had the following conversation:

> "We together have 28 legs," said the first octopus. "We together have 27 legs," said the second octopus. "We together have 26 legs," said the third octopus. "We together have 25 legs," said the fourth octopus.

Question: Which of the four servants told the truth? *Source:* Puzzle Prime, "King Octopus and His Servants."

Rules / Assumptions

Before solving the puzzle, we translate its natural-language statements into a precise mathematical model. The following assumptions describe the model used in this solution.

1. There are exactly four servants in the conversation.
2. Each servant has exactly one of the following numbers of legs:

$$6, \qquad 7, \qquad \text{or} \qquad 8.$$

3. When a servant says, "We together have n legs," the word "we" refers to the same group of all four servants in the conversation. Therefore, every servant is making a claim about the same total number of legs.
4. A servant with 7 legs always makes a false statement.
5. A servant with 6 or 8 legs always makes a true statement.
6. Each quoted sentence is treated as a complete mathematical statement. For example, "We together have 28 legs" means that the total is exactly 28, not approximately 28 and not at least 28.

7. The servants are counted once each, and "legs" has its ordinary meaning. We do not introduce missing legs, artificial legs, unmentioned servants, or other information not present in the problem.

8. The four statements refer to the same group at the same time. The total number of legs does not change during the conversation.

Let x_i be the number of legs belonging to servant i, where

$$x_i \in \{6, 7, 8\} \qquad \text{for } i \in \{1, 2, 3, 4\}.$$

Let the total number of legs be

$$T = x_1 + x_2 + x_3 + x_4.$$

The four servants are therefore asserting, in order,

$$T = 28, \qquad T = 27, \qquad T = 26, \qquad T = 25.$$

For each servant i, the truth value of that servant's statement is determined by the servant's number of legs:

$$x_i = 7 \quad \Longleftrightarrow \quad \text{servant } i\text{'s statement is false,}$$

and

$$x_i \in \{6, 8\} \quad \Longleftrightarrow \quad \text{servant } i\text{'s statement is true.}$$

These assumptions do not tell us in advance which servant is truthful. They only establish the rules that a proposed solution must satisfy.

Work

Let x_i be the number of legs belonging to servant i, and let

$$T = x_1 + x_2 + x_3 + x_4$$

be the total number of legs belonging to the four servants. The four statements are:

$$S_1 : T = 28,$$
$$S_2 : T = 27,$$
$$S_3 : T = 26,$$
$$S_4 : T = 25.$$

A servant with 7 legs lies. A servant with 6 or 8 legs tells the truth.

Claim. *The second servant told the truth. The second servant has 6 legs, the other three servants each have 7 legs, and together the four servants have 27 legs.*

Proof. We first show that at least one servant must be telling the truth. Suppose, for the sake of contradiction, that all four servants are lying. Every liar must have 7 legs, so all four servants would have 7 legs. Their total would therefore be

$$T = 7 + 7 + 7 + 7 = 28.$$

But the first servant claimed that $T = 28$. The first servant's statement would then be true, contradicting our assumption that all four servants were lying. Therefore, at least one servant tells the truth. Next, observe that the four servants claim four different totals:

$$28, \qquad 27, \qquad 26, \qquad 25.$$

The group has only one actual total T, so no two of these different equalities can both be true. Therefore, at most one servant tells the truth. Since at least one servant tells the truth and at most one servant tells the truth, exactly one servant tells the truth. The other three servants lie. Each lying servant must have 7 legs, so the three liars contribute

$$3 \cdot 7 = 21$$

legs. The one truthful servant must have either 6 or 8 legs. If the truthful servant has 6 legs, then

$$T = 21 + 6 = 27.$$

If the truthful servant has 8 legs, then

$$T = 21 + 8 = 29.$$

A truthful servant's statement must equal the actual total. None of the four servants claimed that $T = 29$, but the second servant claimed that

$$T = 27.$$

Therefore, the truthful servant must have 6 legs, the actual total must be 27, and the truthful servant must be the second servant. Thus,

$$(x_1, x_2, x_3, x_4) = (7, 6, 7, 7),$$

and the second servant is the only servant who told the truth. □

Check Your Answer

Our proposed solution is

$$(x_1, x_2, x_3, x_4) = (7, 6, 7, 7).$$

We will check this answer in three ways:

1. verify the arithmetic and every servant's statement directly;
2. review the major steps of the logical proof;
3. compare the result with an exhaustive computer search.

Direct Verification

First, compute the total number of legs:

$$T = 7 + 6 + 7 + 7 = 27.$$

We now compare each servant's statement with the actual total and with the rule determined by that servant's number of legs.

Servant	Legs	Claim	Actual truth value	Required behavior
1	7	$T = 28$	False	Must lie
2	6	$T = 27$	True	Must tell truth
3	7	$T = 26$	False	Must lie
4	7	$T = 25$	False	Must lie

Every row is consistent with the rules:

- Servant 1 has 7 legs and makes a false statement.
- Servant 2 has 6 legs and makes a true statement.
- Servant 3 has 7 legs and makes a false statement.
- Servant 4 has 7 legs and makes a false statement.

Therefore, the assignment

$$(7, 6, 7, 7)$$

satisfies every condition in the puzzle.

Verification of the Logical Proof

The proof relied on the following steps.

1. If all four servants were lying, they would all have 7 legs.
2. Four servants with 7 legs would have
$$4 \cdot 7 = 28$$
legs in total.
3. If the total were 28, then the first servant's statement would be true. This contradicts the assumption that all four servants were lying.
4. Therefore, at least one servant tells the truth.
5. The four servants claim four different totals:
$$28, \qquad 27, \qquad 26, \qquad 25.$$
Since all four statements refer to the same fixed total T, no two of these statements can both be true.
6. Therefore, at most one servant tells the truth.
7. If at least one servant tells the truth and at most one servant tells the truth, then exactly one servant tells the truth.
8. The three liars must each have 7 legs, contributing
$$3 \cdot 7 = 21$$
legs.

9. The truthful servant must have either 6 or 8 legs. Therefore, the only possible totals are

$$21 + 6 = 27$$

and

$$21 + 8 = 29.$$

10. No servant claimed that the total was 29. The second servant claimed that the total was 27.
11. Therefore, the truthful servant has 6 legs, the total is 27, and the second servant is the unique truth teller.

Each step follows from the puzzle's stated rules, ordinary arithmetic, or a previously established step.

Verification by Exhaustive Search

A separate Python program checked every possible assignment of leg counts. Since each of the four servants may have 6, 7, or 8 legs, the program examined

$$3^4 = 81$$

assignments. For each assignment, the program:

1. calculated the total number of legs;
2. evaluated each of the four numerical statements;
3. determined the truth value required by each servant's number of legs;
4. rejected the assignment if any servant's actual statement disagreed with the required behavior.

The search produced the following summary:

```
Assignments checked: 81 Solutions found: 1 Solution legs: (7, 6, 7, 7) Total legs: 27 Truthful
```

The full search results are stored in the accompanying files:

- `king_octopus_bruteforce_results.txt`
- `king_octopus_bruteforce_results.html`

The program that generates both reports is stored in:

`king_octopus_bruteforce_report_generator.py`

The computer search agrees with the logical proof and finds no second solution.

What the Computer Check Does and Does Not Establish

The exhaustive search establishes that exactly one of the 81 assignments in our mathematical model satisfies all of the rules encoded in the program. However, the computer cannot determine whether we translated the original English problem correctly. A programming error or an incorrect assumption could cause a program to search the wrong mathematical model perfectly. For that reason, the logical proof and the computer search serve different purposes:

- The logical proof explains why the answer is forced.

- The direct verification confirms that the proposed answer satisfies every rule.
- The exhaustive search confirms that no other assignment in the model also works.

All three methods agree. Therefore, our answer passes the check:

$$\boxed{\text{The second servant told the truth.}}$$

One-Sentence Summary

The puzzle's rules force the unique arrangement

$$(x_1, x_2, x_3, x_4) = (7, 6, 7, 7),$$

so the four servants have 27 legs in total and the second servant is the only servant who told the truth.

Strongest Disagreement

The strongest objection is that the exhaustive computer search is only as trustworthy as the mathematical model encoded in the program. A program may inspect all 81 assignments correctly while still answering the wrong question if we incorrectly interpret words such as "we," "always," or "together." This objection does not defeat the solution because the assumptions are stated explicitly, the logical proof does not depend on the Python program, and the proposed arrangement is verified directly against every condition in the puzzle. The logical proof, direct check, and exhaustive search all reach the same conclusion.

Included Computational Artifacts

This PDF contains the original computational artifacts used to verify the solution. Select a filename below to extract that file from the PDF.

- **Python report generator:**
- **Interactive HTML results:**
- **Plain-text results:**

The attached Python file is the reproducible recipe. The HTML file is the polished, interactive report. The text file is a portable record of the complete program output. Some web-browser PDF viewers do not display embedded attachments. If an attachment does not appear when the PDF is viewed in a browser, download the PDF and open it in a desktop PDF reader that supports file attachments.

Python Source Code

The complete source code for the exhaustive search and report generator is printed below. The original executable file is also attached to this PDF.

```
from html import escape
from itertools import product
from pathlib import Path


CLAIMS = (28, 27, 26, 25)

TEXT_REPORT = Path("king_octopus_bruteforce_results.txt")
HTML_REPORT = Path("king_octopus_bruteforce_results.html")


def expected_truth_from_legs(leg_count):
    """
    Return the truth value required by the servant's leg count.

    Six-legged and eight-legged servants tell the truth.
    Seven-legged servants lie.
    """
    return leg_count in (6, 8)


def truth_word(value):
    """Convert a Boolean value to a readable word."""
    return "TRUE" if value else "FALSE"


def evaluate_assignment(option_number, legs):
    """
    Evaluate one possible assignment of leg counts.

    Return a dictionary containing the total, servant-level results,
    and the overall result.
    """
    total = sum(legs)
    servant_results = []
    assignment_works = True

    for servant_number, (leg_count, claimed_total) in enumerate(
        zip(legs, CLAIMS),
        start=1,
    ):
        statement_is_true = total == claimed_total
        required_truth = expected_truth_from_legs(leg_count)
        servant_passes = statement_is_true == required_truth

        if servant_passes:
            explanation = (
                "The statement's truth value agrees with the rule "
                "for this servant's leg count."
            )
        elif leg_count == 7 and statement_is_true:
            explanation = (
                "A seven-legged servant must lie, but this servant's "
                "statement is true."
            )
        else:
            explanation = (
                "A six-legged or eight-legged servant must tell the "
                "truth, but this servant's statement is false."
            )

        if not servant_passes:
            assignment_works = False

        servant_results.append(
            {
```

```
                "servant_number": servant_number,
                "leg_count": leg_count,
                "claimed_total": claimed_total,
                "actual_total": total,
                "statement_is_true": statement_is_true,
                "required_truth": required_truth,
                "passes": servant_passes,
                "explanation": explanation,
            }
        )

    return {
        "option_number": option_number,
        "legs": legs,
        "total": total,
        "servants": servant_results,
        "works": assignment_works,
    }


def build_text_report(results):
    """Create a plain-text report suitable for inclusion in LaTeX."""
    lines = []

    lines.append("KING OCTOPUS BRUTE-FORCE SEARCH")
    lines.append("=" * 72)
    lines.append("")
    lines.append("Rules:")
    lines.append("  * Each servant has 6, 7, or 8 legs.")
    lines.append("  * A seven-legged servant always lies.")
    lines.append("  * A six-legged or eight-legged servant always tells the truth.")
    lines.append("  * The four claimed totals are 28, 27, 26, and 25.")
    lines.append("")
    lines.append(f"Assignments checked: {len(results)}")
    lines.append("")

    for result in results:
        status = "WORKS" if result["works"] else "DOES NOT WORK"

        lines.append("-" * 72)
        lines.append(
            f"OPTION {result['option_number']:02d}: "
            f"legs = {result['legs']}"
        )
        lines.append(f"Actual total: {result['total']}")
        lines.append(f"Overall result: {status}")
        lines.append("")

        for servant in result["servants"]:
            servant_status = "PASS" if servant["passes"] else "FAIL"

            lines.append(
                f"  Servant {servant['servant_number']}: "
                f"{servant['leg_count']} legs"
            )
            lines.append(
                f"    Claims that the total is "
                f"{servant['claimed_total']}."
            )
            lines.append(
                f"    The actual total is "
                f"{servant['actual_total']}."
            )
            lines.append(
                f"    Statement is actually: "
                f"{truth_word(servant['statement_is_true'])}"
            )
            lines.append(
```

```
                f"    Statement must be: "
                f"{truth_word(servant['required_truth'])}"
            )
            lines.append(f"    Result: {servant_status}")
            lines.append(
                f"    Explanation: {servant['explanation']}"
            )
            lines.append("")

    solutions = [result for result in results if result["works"]]

    lines.append("=" * 72)
    lines.append("FINAL RESULTS")
    lines.append("=" * 72)
    lines.append(f"Assignments checked: {len(results)}")
    lines.append(f"Solutions found: {len(solutions)}")
    lines.append("")

    if not solutions:
        lines.append("No assignment satisfies all of the rules.")
    else:
        for result in solutions:
            truthful_servants = [
                servant["servant_number"]
                for servant in result["servants"]
                if servant["statement_is_true"]
            ]

            lines.append(f"Solution legs: {result['legs']}")
            lines.append(f"Total legs: {result['total']}")
            lines.append(
                "Truthful servant(s): "
                + ", ".join(str(number) for number in truthful_servants)
            )
            lines.append("")

    return "\n".join(lines)


def build_html_report(results):
    """Create a polished HTML report suitable for a web browser."""
    solutions = [result for result in results if result["works"]]

    option_sections = []

    for result in results:
        option_class = "working-option" if result["works"] else "failed-option"
        option_status = "WORKS" if result["works"] else "DOES NOT WORK"

        servant_rows = []

        for servant in result["servants"]:
            result_class = "pass" if servant["passes"] else "fail"
            result_word = "PASS" if servant["passes"] else "FAIL"

            servant_rows.append(
                f"""
                <tr>
                    <td>{servant["servant_number"]}</td>
                    <td>{servant["leg_count"]}</td>
                    <td>{servant["claimed_total"]}</td>
                    <td>{servant["actual_total"]}</td>
                    <td>{truth_word(servant["statement_is_true"])}</td>
                    <td>{truth_word(servant["required_truth"])}</td>
                    <td class="{result_class}">{result_word}</td>
                    <td>{escape(servant["explanation"])}</td>
                </tr>
                """
```

```
            )

        servant_table = "\n".join(servant_rows)

        option_sections.append(
            f"""
            <details class="option {option_class}"
                     {"open" if result["works"] else ""}>
                <summary>
                    Option {result["option_number"]:02d}:
                    legs = {escape(str(result["legs"]))},
                    total = {result["total"]},
                    {option_status}
                </summary>

                <div class="table-container">
                    <table>
                        <thead>
                            <tr>
                                <th>Servant</th>
                                <th>Legs</th>
                                <th>Claim</th>
                                <th>Actual Total</th>
                                <th>Statement</th>
                                <th>Required</th>
                                <th>Result</th>
                                <th>Explanation</th>
                            </tr>
                        </thead>
                        <tbody>
                            {servant_table}
                        </tbody>
                    </table>
                </div>
            </details>
            """
        )

    solution_items = []

    for result in solutions:
        truthful_servants = [
            servant["servant_number"]
            for servant in result["servants"]
            if servant["statement_is_true"]
        ]

        solution_items.append(
            f"""
            <li>
                Legs: <code>{escape(str(result["legs"]))}</code><br>
                Total: <strong>{result["total"]}</strong><br>
                Truthful servant(s):
                <strong>
                    {escape(", ".join(str(n) for n in truthful_servants))}
                </strong>
            </li>
            """
        )

    if solution_items:
        solution_html = "<ul>" + "\n".join(solution_items) + "</ul>"
    else:
        solution_html = "<p>No assignment satisfies all rules.</p>"

    all_options_html = "\n".join(option_sections)

    return f"""<!DOCTYPE html>
```

```
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>King Octopus Brute-Force Results</title>

    <style>
        :root {{
            color-scheme: light;
            font-family:
                Inter, Segoe UI, Arial, sans-serif;
            line-height: 1.5;
        }}

        body {{
            max-width: 1200px;
            margin: 0 auto;
            padding: 2rem;
            color: #172033;
            background: #f4f7fb;
        }}

        h1, h2 {{
            color: #12355b;
        }}

        .summary {{
            padding: 1.25rem;
            margin-bottom: 1.5rem;
            border-left: 6px solid #2463a7;
            border-radius: 8px;
            background: white;
            box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
        }}

        code {{
            padding: 0.15rem 0.35rem;
            border-radius: 4px;
            background: #e8eef6;
        }}

        details {{
            margin: 0.75rem 0;
            border: 1px solid #c8d1dc;
            border-radius: 8px;
            background: white;
            overflow: hidden;
        }}

        summary {{
            padding: 1rem;
            cursor: pointer;
            font-weight: 700;
        }}

        .working-option {{
            border: 3px solid #218739;
        }}

        .working-option summary {{
            color: #145c26;
            background: #eaf7ed;
        }}

        .failed-option summary {{
            background: #f8fafc;
        }}

```

```
        .table-container {{
            overflow-x: auto;
            padding: 1rem;
        }}

        table {{
            width: 100%;
            border-collapse: collapse;
            font-size: 0.92rem;
        }}

        th, td {{
            padding: 0.65rem;
            border: 1px solid #d8dee8;
            text-align: left;
            vertical-align: top;
        }}

        th {{
            color: white;
            background: #24496f;
        }}

        tr:nth-child(even) {{
            background: #f6f8fb;
        }}

        .pass {{
            color: #126b2a;
            font-weight: 700;
        }}

        .fail {{
            color: #a11b1b;
            font-weight: 700;
        }}

        footer {{
            margin-top: 2rem;
            color: #526173;
            font-size: 0.9rem;
        }}
    </style>
</head>

<body>
    <h1>King Octopus Brute-Force Search</h1>

    <section class="summary">
        <h2>Search Summary</h2>

        <p>
            The program checked all
            <strong>{len(results)}</strong>
            possible assignments.
        </p>

        <p>
            Number of solutions:
            <strong>{len(solutions)}</strong>
        </p>

        <h2>Valid Solution</h2>
        {solution_html}
    </section>

    <section>
        <h2>Every Assignment</h2>
```

```
        <p>
            Select an option to display the detailed reason that it
            works or fails. The valid option is expanded automatically.
        </p>

        {all_options_html}
    </section>

    <footer>
        Generated by king_octopus_bruteforce_report_generator.py
    </footer>
</body>
</html>
"""


def main():
    results = [
        evaluate_assignment(option_number, legs)
        for option_number, legs in enumerate(
            product((6, 7, 8), repeat=4),
            start=1,
        )
    ]

    text_report = build_text_report(results)
    html_report = build_html_report(results)

    TEXT_REPORT.write_text(text_report, encoding="utf-8")
    HTML_REPORT.write_text(html_report, encoding="utf-8")

    solutions = [result for result in results if result["works"]]

    print("King Octopus brute-force search complete.")
    print(f"Assignments checked: {len(results)}")
    print(f"Solutions found: {len(solutions)}")
    print()
    print(f"Text report: {TEXT_REPORT.resolve()}")
    print(f"HTML report: {HTML_REPORT.resolve()}")

    for result in solutions:
        truthful_servants = [
            servant["servant_number"]
            for servant in result["servants"]
            if servant["statement_is_true"]
        ]

        print()
        print(f"Solution legs: {result['legs']}")
        print(f"Total legs: {result['total']}")
        print(f"Truthful servant(s): {truthful_servants}")


if __name__ == "__main__":
    main()
```

Included Computational Artifacts

This PDF contains the original computational artifacts used to verify the solution. Select a filename below to extract that file from the PDF.

- **Python report generator:** king_octopus_bruteforce_report_generator.py

- **Interactive HTML results:** king_octopus_bruteforce_results.html
- **Plain-text results:** king_octopus_bruteforce_results.txt

The attached Python file is the reproducible recipe. The HTML file is the polished, interactive report. The text file is a portable record of the complete program output. Some browser-based PDF viewers do not expose embedded attachments. If an attachment is not available in the browser, download the PDF and open it in a desktop PDF reader that supports PDF file attachments.